Python Programming Cheatsheet – CDAC CCEE Exam

Sessions 1–7: Complete Quick Reference Guide

SESSION 1: BASICS, SYNTAX, DATA TYPES, CONTROL FLOW

Data Types & Variables

Type	Example	Mutable?
int	x = 42	—
float	pi = 3.14	—
str	name = "CDAC"	No
bool	flag = True	—
list	nums = [1, 2, 3]	Yes
tuple	point = (1, 2)	No
dict	d = {"a": 1}	Yes

Variable Declaration & Assignment

```
x = 10 # single assignment
x, y, z = 1, 2, 3 # multiple assignment (unpacking)
x = y = z = 0 # chained assignment
```

Operators

Arithmetic: +, -, *, / (float), // (floor), % (modulo), ** (power)
Comparison: ==, !=, <, >, <=, >=
Logical: and, or, not
Membership: in, not in
Assignment: =, +=, -=, *=, /=, //=, %=, **=

Critical trick: In Python 3, / always returns float. Use // for integer division.

```
5 / 2 # 2.5
5 // 2 # 2
```

Input/Output

```
x = input("Enter value: ") # returns string
x = int(input("Enter number: ")) # convert to int
print("Sum:", a + b) # print with multiple args
```

Control Structures: if/elif/else

```
if condition1:
# block 1
elif condition2:
# block 2
else:
# block 3
```

Key: Only the first true branch executes. Use `elif` for multiple conditions.

Loops: for and while

For loop (over sequence or range):

```
for i in range(1, 5): # i = 1, 2, 3, 4
print(i)

for ch in "CDAC":
print(ch)
```

While loop (condition-based):

```
i = 0
while i < 5:
print(i)
i += 1
```

Loop Control Statements

Statement	Effect
`break`	Exit nearest enclosing loop immediately
`continue`	Skip rest of current iteration; go to next
`pass`	Do nothing; placeholder for future code

```
for i in range(5):
if i == 2:
continue # skips i=2, prints 0,1,3,4
if i == 4:
break # exits loop when i=4
print(i)

for i in range(3):
pass # does nothing, loop runs normally
```

Trick: `break`/`continue` affect only the innermost loop they are in.

SESSION 2: STRINGS & TUPLES

String Basics

- **Immutable:** Cannot change characters in place.
- **Indexing:** s[0] (first), s[-1] (last), s[-2] (second-to-last).
- **Length:** len(s).

```
s = "CDAC"
s[0] # 'C'
s[-1] # 'C'
len(s) # 4
```

String Slicing

```
s = "PYTHON"
s[0:2] # 'PY' (start inclusive, end exclusive)
s[2:5] # 'THO'
s[:3] # 'PYT' (start from beginning)
s[3:] # 'HON' (go to end)
s[::2] # 'PTO' (every 2nd char)
s[::-1] # 'NOHTYP' (reversed)
```

String Operations

```
"CD" + "AC" # concatenation → "CDAC"
"ab" * 3 # repetition → "abababab"
"D" in "CDAC" # membership → True
"CDAC".upper() # → "CDAC"
"CDAC".lower() # → "cdac"
" CDAC ".strip() # → "CDAC"
"a,b,c".split(",") # → ["a", "b", "c"]
" ".join(["a", "b"]) # → "a b"
```

String Formatting

```
name, score = "Rahul", 95
```

f-strings (Python 3.6+) – preferred

```
f"Student {name} scored {score}%"
```

.format() method

"Student {} scored {}%".format(name, score)

% formatting (older)

"Student %s scored %d%%" % (name, score)

Tuples: Immutable Sequences

t = (1, 2, 3)
t[0] # 1 (indexing)
t[1:3] # (2, 3) (slicing)
2 in t # True (membership)

Single-element tuple needs comma

s = (5,) # tuple
x = (5) # NOT a tuple, just int!

Unpacking

a, b, c = t # a=1, b=2, c=3

Key: Tuples are immutable. You cannot do `t[0] = 99`. However, if a tuple contains a mutable object (like a list), that object can still be modified:
t = (1, [2, 3])
t[1].append(4) # allowed → t = (1, [2, 3, 4])
t[0] = 99 # NOT allowed → TypeError

Tuple Operations

t1 + t2 # concatenation
t * 3 # repetition
len(t) # length
t.count(2) # count occurrences of 2
t.index(3) # index of first 3

SESSION 3: DICTIONARIES

Dictionary Basics

- **Key-value pairs:** `{key: value}`
- **Keys must be unique & immutable** (int, str, tuple)
- **Values can be anything**
- **Ordered** (preserves insertion order in Python 3.7+)

```
d = {"name": "Rahul", "age": 25, "marks": 85}
d["name"] # "Rahul"
d.get("age") # 25
d.get("city", "N/A") # "N/A" (default if key missing)
```

Modifying Dictionaries

```
d["city"] = "Pune" # add/update key
d.update({"marks": 95}) # update from another dict
del d["age"] # delete key
d.pop("marks") # remove & return value
d.popitem() # remove arbitrary (last inserted) pair
d.clear() # remove all items
```

Trap: `del d["missing"]` and `d.pop("missing")` raise `KeyError` unless you provide a default.

Dictionary Methods

Method	Returns
`d.keys()`	All keys
`d.values()`	All values
`d.items()`	All (key, value) pairs

```
for k, v in d.items():
print(k, v)
```

Dictionary Comprehension

```
squares = {x: x*x for x in range(1, 4)} # {1: 1, 2: 4, 3: 9}
evens = {x: x*x for x in range(6) if x % 2 == 0} # {0: 0, 2: 4, 4: 16}
doubled = {k: v*2 for k, v in d.items()} # double all values
```

SESSION 4: LISTS

List Basics

- **Mutable:** Can change, add, remove elements.
- **Ordered:** Preserves insertion order.
- **Can store mixed types & duplicates.**

```
l = [1, "CDAC", 3.14, True]
l[0] # 1
l[-1] # True (last)
l[1:3] # ["CDAC", 3.14]
len(l) # 4
```

List Operations

[1, 2] + [3, 4] # [1, 2, 3, 4]
[0] * 3 # [0, 0, 0]
3 in [1, 2, 3] # True
[1, 2, 3].count(2) # 1
[1, 2, 2, 3].index(2) # 1 (first occurrence)

Adding Elements

l = [1, 2]
l.append(3) # [1, 2, 3] – adds at end
l.insert(1, 99) # [1, 99, 2, 3] – inserts at index 1
l.extend([4, 5]) # [1, 99, 2, 3, 4, 5] – merges list

Trap: append vs extend

l2 = [1, 2]
l2.append([3, 4]) # [1, 2, [3, 4]] – single element added
l3 = [1, 2]
l3.extend([3, 4]) # [1, 2, 3, 4] – merged

Removing Elements

l = [1, 2, 3, 2]
del l[1] # [1, 3, 2] – remove by index
l.pop() # removes & returns last element
l.pop(0) # removes & returns first element
l.remove(2) # [1, 3] – removes first matching value
l.clear() # [] – removes all

Trap: `remove()` raises `ValueError` if value not found.

List Slicing

l = [10, 20, 30, 40, 50]
l[1:4] # [20, 30, 40]
l[:3] # [10, 20, 30]
l[2:] # [30, 40, 50]
l[::2] # [10, 30, 50]
l[::-1] # [50, 40, 30, 20, 10] (reversed)

List Comprehension

nums = [1, 2, 3, 4]
squares = [x*x for x in nums] # [1, 4, 9, 16]*
*evens_sq = [x*x for x in nums if x % 2 == 0] # [4, 16]

Joining & Splitting

```
words = ["CDAC", "Python", "CCEE"]
s = "-".join(words) # "CDAC-Python-CCEE"

text = "a,b,c"
parts = text.split(",") # ["a", "b", "c"]
```

Trap: join() is called on separator, not the list

```
"–".join(["a", "b"]) # correct
["a", "b"].join("–") # ERROR
```

List vs Tuple vs String

Property	List	Tuple	String
Mutable?	Yes	No	No
Syntax	[...]	(...)	"..." or '...'
Indexing	✓	✓	✓
Slicing	✓	✓	✓
Can be dict key?	No	Yes*	Yes

*Only if tuple contains immutable elements.

SESSION 5 & 6: FUNCTIONS, LAMBDAS, MODULES, PICKLING

Function Definition & Calling

```
def add(a, b):
return a + b

result = add(2, 3) # 5
```

Return: If no `return`, function returns `None`.

Function Arguments

Positional:

```
def power(a, b):
return a ** b

power(2, 3) # a=2, b=3
```

Default values:
def greet(name="Guest"):
return f"Hello, {name}"

greet() # "Hello, Guest"
greet("Rahul") # "Hello, Rahul"

Keyword (named) arguments:
greet(name="Ajay") # "Hello, Ajay"
power(b=3, a=2) # 8 (order doesn't matter)

Variable-length (*args, **kwargs):
def demo(*args, **kwargs):
print(args) # tuple of extra positional args
print(kwargs) # dict of extra keyword args

demo(1, 2, 3, name="Test", age=25)

args = (1, 2, 3)

kwargs = {'name': 'Test', 'age': 25}

Order rule: positional → default → *args → keyword-only → **kwargs

Global vs Local Variables

x = 10 # global

def f():
x = 5 # local (shadows global)
print(x) # 5

f()
print(x) # 10 (global unchanged)

Modify global inside function

count = 0

def inc():
global count
count += 1

Trap: Without global, assignment inside function creates local variable.

Lambda (Anonymous Functions)

```
sq = lambda x: x * x
sq(5) # 25

add = lambda a, b: a + b
add(2, 3) # 5
```

Use: Short, throwaway functions (often with `map`, `filter`, etc.).

map() Function

```
nums = [1, 2, 3]
doubled = list(map(lambda x: x * 2, nums)) # [2, 4, 6]
```

Alternative: list comprehension (often preferred)

```
doubled = [x * 2 for x in nums]
```

Modules & Importing

```
import math
math.sqrt(16) # 4.0

from math import sqrt
sqrt(16) # 4.0

from math import sqrt as square_root
square_root(16) # 4.0
```

`__name__ == "__main__"`: Runs only when file executed directly, not imported.

```
if __name__ == "__main__":
print("Running as main script")
```

Pickling (Serialization)

Convert Python objects to byte stream (save to file or send over network).

```
import pickle

data = {"name": "CDAC", "nums": [1, 2, 3]}
```

Write (pickle) to file

```
with open("data.pkl", "wb") as f:
pickle.dump(data, f)
```

Read (unpickle) from file

with open("data.pkl", "rb") as f:
loaded = pickle.load(f)

Caution: Unpickling untrusted data is unsafe (can execute arbitrary code).

SESSION 7: TUPLES (EXTENDED)

Tuple Characteristics

- **Immutable:** Cannot change elements after creation.
- **Hashable:** Can be used as dictionary keys (if elements are hashable).
- **Ordered:** Preserve insertion order.

t = (1, 2, 3)
t[0] # 1
t[0] = 99 # TypeError: tuple does not support item assignment

Single-Element Tuple (Critical MCQ Trap!)

single = (5,) # tuple with one element
NOT_tuple = (5) # just an int (parentheses don't make it tuple)

Tuple Unpacking

x, y = (10, 20) # x=10, y=20
a, *rest = (1, 2, 3, 4) # a=1, rest=[2, 3, 4]

Tuple Operations

t1 = (1, 2)
t2 = (3, 4)
t1 + t2 # (1, 2, 3, 4)
t1 * 3 # (1, 2, 1, 2, 1, 2)
len(t1) # 2
t1.count(1) # 1
t1.index(2) # 1
2 in t1 # True

Mutable Objects Inside Tuples

t = (1, [2, 3])
t[1].append(4) # allowed! → t = (1, [2, 3, 4])
t[0] = 99 # NOT allowed → TypeError

The tuple itself is immutable, but objects inside it are not.

KEY FORMULAS & COMMON TRICKS

Division Operators

```
7 / 2 # 3.5 (true division, always float)
7 // 2 # 3 (floor division, int)
7 % 2 # 1 (modulo / remainder)
```

String vs List Comparison

```
"hello" == ['h', 'e', 'l', 'l', 'o'] # False (different types)
list("hello") # ['h', 'e', 'l', 'l', 'o']
"".join(['h', 'e', 'l', 'l', 'o']) # "hello"
```

`==` vs `is`

```
l1 = [1, 2]
l2 = [1, 2]
l1 == l2 # True (same content)
l1 is l2 # False (different objects)
```

Default Mutable Argument Trap

Avoid this:

```
def add_item(item, lst=[]):
lst.append(item)
return lst
```

All calls share the same list!

range() Function

```
range(5) # 0, 1, 2, 3, 4
range(1, 5) # 1, 2, 3, 4 (start inclusive, stop exclusive)
range(0, 10, 2) # 0, 2, 4, 6, 8 (step = 2)
range(5, 0, -1) # 5, 4, 3, 2, 1 (reverse)
```

CCEE MCQ STRATEGY

1. **Output-based:** Dry-run code mentally or on paper.
2. **Indexing/Slicing:** Remember 0-based, start inclusive, end exclusive.
3. **Immutability tricks:** String/tuple cannot be modified in place; list can.
4. **Operator precedence:** `and` before `or`, exponentiation before multiplication.
5. **Loop control:** `break` exits loop; `continue` skips iteration; `pass` does nothing.
6. **Type errors:** Understand when things are mutable vs immutable.
7. **Variable scope:** Global vs local; `global` keyword changes assignment behavior.
8. **String methods:** `split`, `join`, `strip`, `upper`, `lower` all return new strings.
9. **List vs tuple:** Same indexing & slicing syntax, but list is mutable.

10. **Dictionary keys:** Must be immutable (int, str, tuple); keys must be unique.

Last Updated: January 2026 | **Target:** CDAC CCEE Exam | **Sessions Covered:** 1–7